Task-1:

HEART DISEASE PREDICTION USING LOGISTIC REGRESSION

Submitted By

Name: A D S Abhishek

Internship: CodeAlpha – Machine Learning Internship

Table of Contents

Abstract

Heart disease is one of the leading causes of death worldwide, making early detection and prediction extremely important. This project aims to develop a machine learning-based system that predicts the possibility of heart disease using patient medical information such as age, sex, chest pain type, blood pressure, cholesterol level, maximum heart rate, and other clinical parameters.

The Heart Disease dataset containing 1025 patient records and 14 attributes was used. Logistic Regression, a supervised machine learning classification algorithm, was applied to predict the presence or absence of heart disease.

The model was trained and evaluated using a train-test split approach. Its performance was measured using classification metrics such as accuracy, precision, recall, and F1-score. The developed system demonstrates how machine learning techniques can be effectively used in healthcare applications to support early disease prediction and assist in clinical decision-making.

Introduction

Heart disease is a major health concern worldwide and is responsible for a large number of deaths every year. Early prediction of heart disease can help healthcare professionals provide timely treatment and preventive measures. Machine learning techniques can analyze medical data and identify patterns that are difficult to observe manually.

This project develops a Heart Disease Prediction System using Logistic Regression to classify whether a patient is likely to have heart disease based on medical parameters.

Problem Statement

To develop a machine learning-based system that predicts whether a patient is likely to have heart disease or not based on medical attributes such as age, sex, chest pain type, blood pressure, cholesterol level, maximum heart rate, and other clinical parameters. The goal is to assist in early diagnosis and support healthcare professionals in making informed decisions.

Objective: Predict the possibility of diseases based on patient data.

- To analyze the Heart Disease dataset.
- To preprocess and visualize medical data.
- To build a Logistic Regression model for disease prediction.
- To evaluate model performance using classification metrics.

Approach: Apply classification techniques to structured medical datasets.
Key Features:

- Use features like symptoms, age, blood test results, etc.
- Algorithms Used: Logistic Regression, Random Forest
- Datasets: Heart disease, Diabetes, Breast Cancer (UCI ML Repository).

Dataset Description

- **Dataset: Heart Disease Dataset (Kaggle/UCI Repository)**
- **Rows (Instances): 1025**
  Columns (Attributes): 14
  Input Features: 13
  Target Variable: target
- **Target:**
  0 → No Heart Disease
  1 → Presence of Heart Disease
- **Features:**
  age, sex, cp, trestbps, chol, fbs, restecg, thalach, exang, oldpeak, slope, ca, thal.
- **Missing Values: 0**
  Data Types: 13 Integer columns and 1 Float column.

Attribute	Description
age	Patient's age (in years)
sex	Gender (0 = Female, 1 = Male)
cp	Type of chest pain
trestbps	Resting blood pressure
chol	Cholesterol level
fbs	Fasting blood sugar (1 = High, 0 = Normal)
restecg	Resting ECG results
thalach	Maximum heart rate achieved
exang	Exercise-induced chest pain (1 = Yes, 0 = No)
oldpeak	ST depression caused by exercise
slope	Slope of the exercise ST segment
ca	Number of major blood vessels

thal	Thalassemia type
target	Heart disease prediction (0 = No Disease, 1 = Disease)

Data Preprocessing

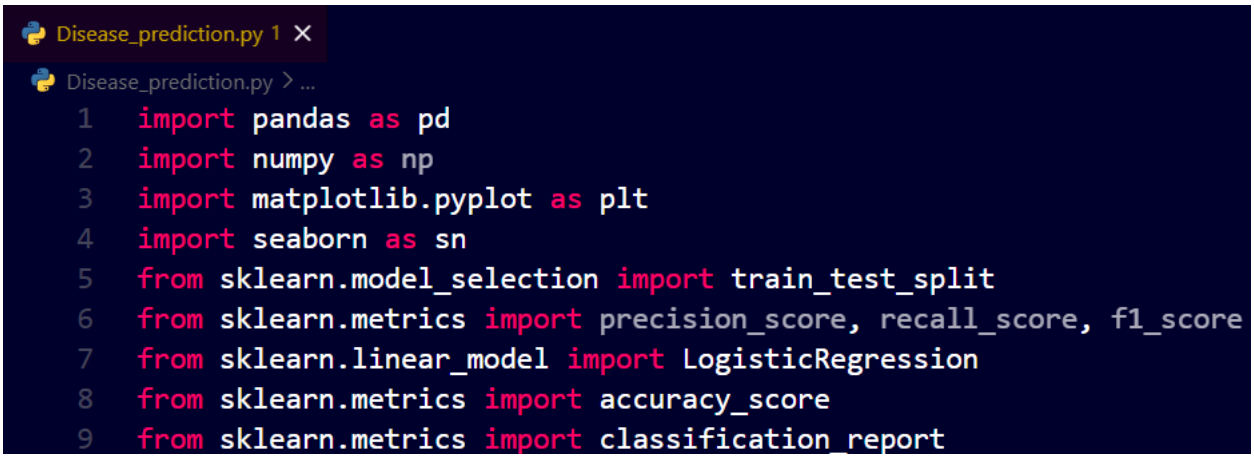
Data preprocessing is an important step in machine learning that involves preparing the dataset for model training and improving data quality.

- **Data inspection using:**

df.head()
df.info()
df.describe()
df.isnull().sum()

- **Missing values: None**
- **Duplicate values: None**
- **Encoding required: No**
- **All features were already numerical and ready for model training.**

Therefore, the dataset was directly divided into input features (X) and target variable (y) and used for training the Logistic Regression model.



These are the required and imported libraries/packages.

Dataset link used from the Kaggle resource: and required link.

https://www.kaggle.com/datasets/johnsmith88/heart-disease-dataset

Exploratory Data Analysis (EDA)

1. Age Distribution Histogram
2. Heart Disease Distribution Count Plot
3. Correlation Heatmap
4. Confusion Matrix Heatmap

Methodology:

Heart Dataset

↓

Data Preprocessing

↓

Exploratory Data Analysis

↓

Feature Selection

↓

Train-Test Split

↓

Logistic Regression Model

↓

Prediction

↓

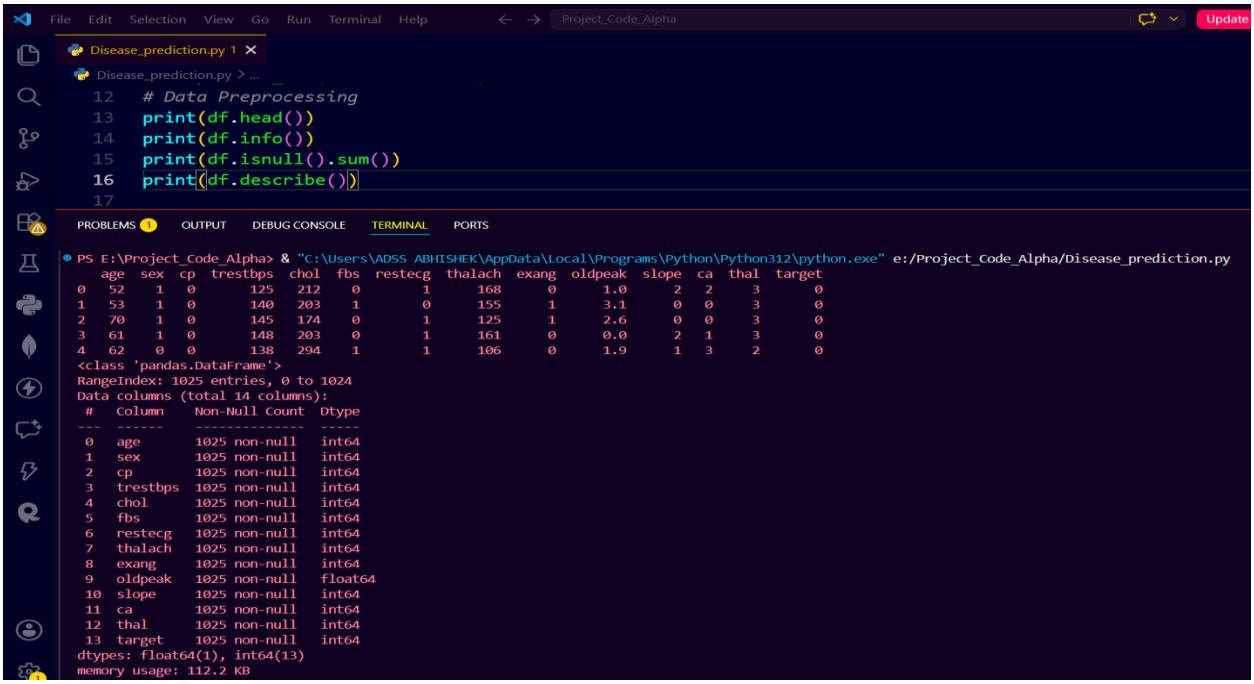
Model Evaluation

↓

Conclusion

Screenshots of the Data and the executed code:

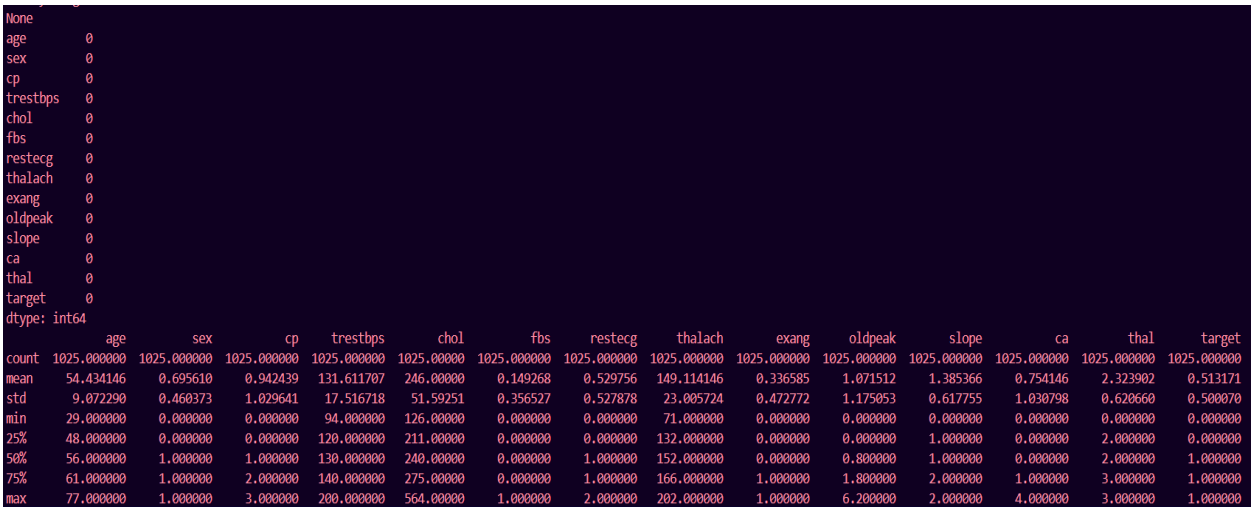
df = pd.read_csv("D:/heart.csv")



```
PS E:\Project_Code_Alpha> & "C:\Users\ADSS ABHISHEK\AppData\Local\Programs\Python\Python312\python.exe" e:/Project_Code_Alpha/Disease_prediction.py
   age  sex  cp  trestbps  chol  fbs  restecg  thalach  exang  oldpeak  slope  ca  thal  target
0   52    1   0       125   212    0        1      168      0      1.0      2   2     3       0
1   53    1   0       140   203    1        0      155      1      3.1      0   0     3       0
2   70    1   0       145   174    0        1      125      1      2.6      0   0     3       0
3   61    1   0       148   203    0        1      161      0      0.0      2   1     3       0
4   62    0   0       138   294    1        1      106      0      1.9      1   3     2       0
<class 'pandas.DataFrame'>
RangeIndex: 1025 entries, 0 to 1024
Data columns (total 14 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   age       1025 non-null   int64
 1   sex       1025 non-null   int64
 2   cp        1025 non-null   int64
 3   trestbps  1025 non-null   int64
 4   chol      1025 non-null   int64
 5   fbs       1025 non-null   int64
 6   restecg   1025 non-null   int64
 7   thalach   1025 non-null   int64
 8   exang     1025 non-null   int64
 9   oldpeak   1025 non-null   float64
 10  slope     1025 non-null   int64
 11  ca        1025 non-null   int64
 12  thal      1025 non-null   int64
 13  target    1025 non-null   int64
dtypes: float64(1), int64(13)
memory usage: 112.2 KB
```

print(df.isnull().sum())

print(df.describe())



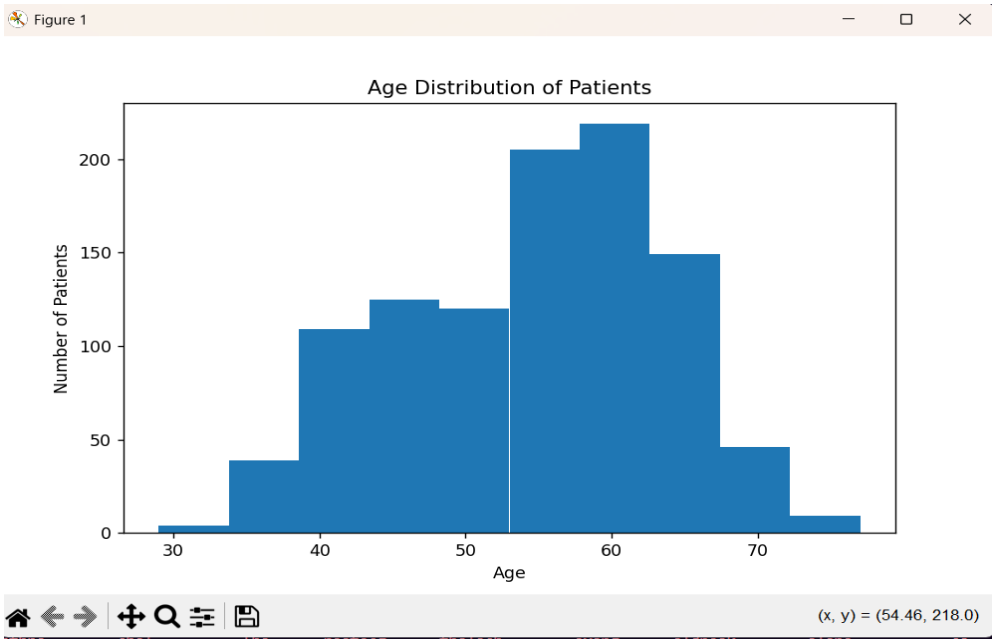
```
None
age         0
sex         0
cp          0
trestbps    0
chol        0
fbs         0
restecg     0
thalach     0
exang       0
oldpeak     0
slope       0
ca          0
thal        0
target      0
dtype: int64
              age         sex          cp    trestbps        chol         fbs     restecg     thalach       exang     oldpeak       slope          ca        thal      target
count  1025.000000  1025.000000  1025.000000  1025.000000  1025.00000  1025.000000  1025.000000  1025.000000  1025.000000  1025.000000  1025.000000  1025.000000  1025.000000  1025.000000
mean     54.434146     0.695610     0.942439   131.611707   246.00000     0.149268     0.529756   149.114146     0.336585     1.071512     1.385366     0.754146     2.323902     0.513171
std       9.072290     0.460373     1.029641    17.516718    51.59251     0.356527     0.527878    23.005724     0.472772     1.175053     0.617755     1.030798     0.620660     0.500070
min      29.000000     0.000000     0.000000    94.000000   126.00000     0.000000     0.000000    71.000000     0.000000     0.000000     0.000000     0.000000     0.000000     0.000000
25%      48.000000     0.000000     0.000000   120.000000   211.00000     0.000000     0.000000   132.000000     0.000000     0.000000     1.000000     0.000000     2.000000     0.000000
50%      56.000000     1.000000     1.000000   130.000000   240.00000     0.000000     1.000000   152.000000     0.000000     0.800000     1.000000     0.000000     2.000000     1.000000
75%      61.000000     1.000000     2.000000   140.000000   275.00000     0.000000     1.000000   166.000000     1.000000     1.800000     2.000000     1.000000     3.000000     1.000000
max      77.000000     1.000000     3.000000   200.000000   564.00000     1.000000     2.000000   202.000000     1.000000     6.200000     2.000000     4.000000     3.000000     1.000000
```

 # Showing the Histogram:

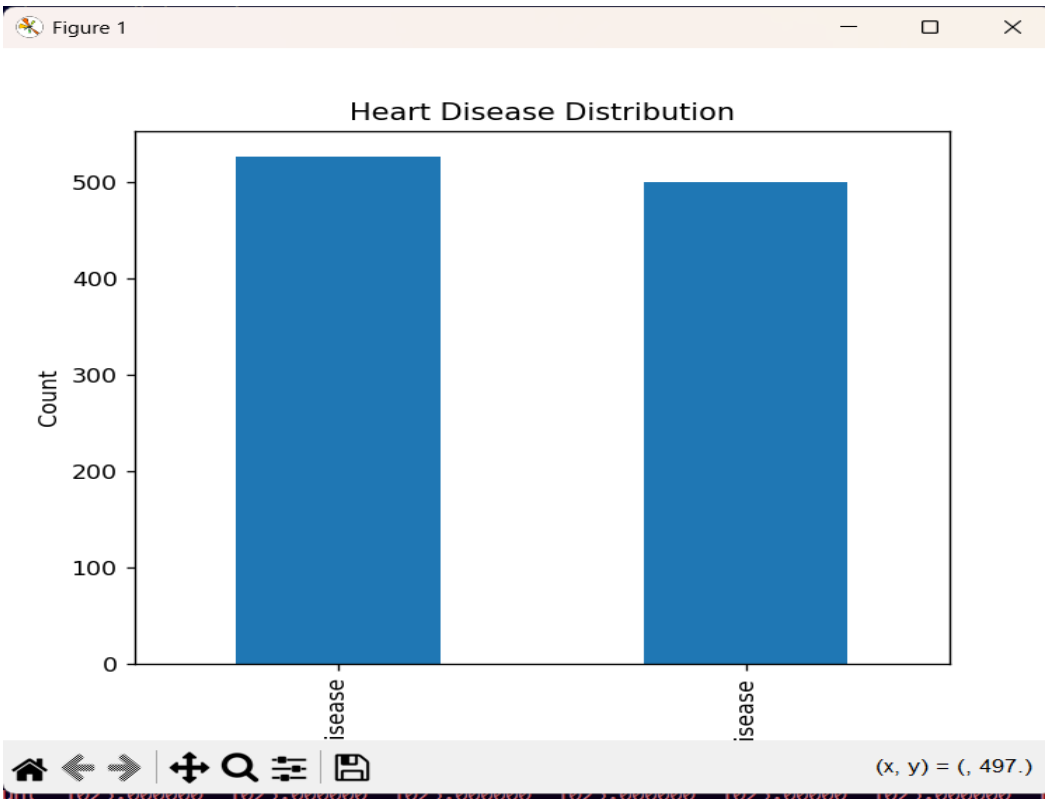
plt.figure(figsize=(8,5))

plt.hist(df['age'], bins=10)

```
plt.xlabel("Age")
plt.ylabel("Number of Patients")
plt.title("Age Distribution of Patients")
plt.show()
```

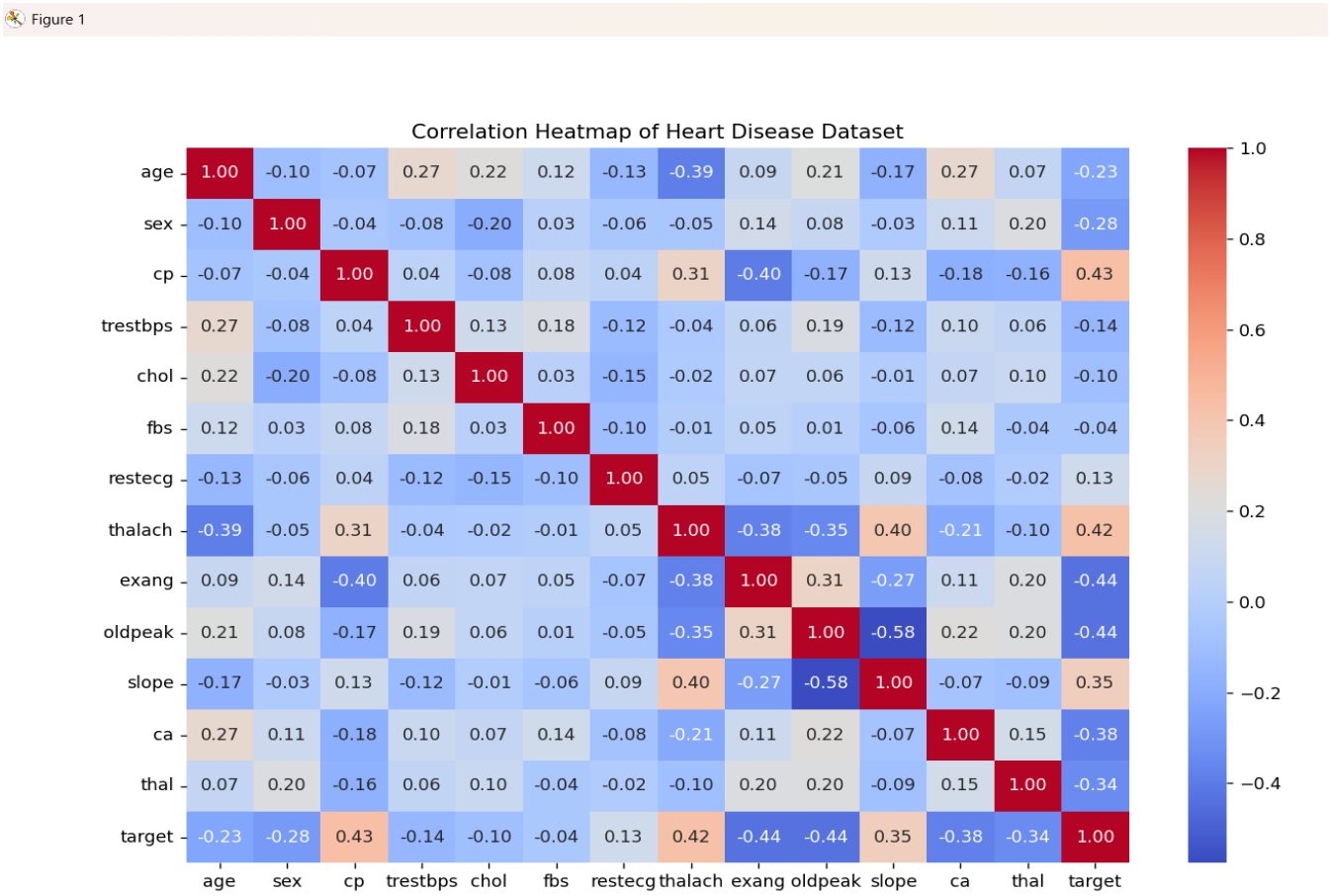



```
# using the Matplotlib: Disease vs No Disease (Count Plot)
# Purpose: Shows the number of patients with and without heart disease.
plt.figure(figsize=(6,5))
df['target'].value_counts().plot(kind='bar')
plt.xlabel("Heart Disease")
plt.ylabel("Count")
plt.title("Heart Disease Distribution")
plt.xticks([0,1], ['No Disease','Disease'])
plt.show().
```




```
# Heatmap Representation:
correlation = df.corr()
plt.figure(figsize=(12,8))
sn.heatmap(
    correlation,
    annot=True,      # Shows correlation values inside each cell
    cmap="coolwarm", # Color scheme
    fmt=".2f"        # Shows values up to 2 decimal places
)
```

plt.title("Correlation Heatmap of Heart Disease Dataset") plt.show().



Feature Selection

```
X = df.drop('target',axis=1)
y = df['target']
```

X = input features

y = output label.

Train-Test Split

```
X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.2,random_state=42)
```

Test_size = 0.2 means

80% → Training.

20% → Testing. And random_state=42 that means we are having the Standard dataset for the Model evaluation and prediction.

Model Building & Prediction

Algorithm Used: Logistic Regression

Train-Test Split:
Training Data = 80%
Testing Data = 20%
Random State = 42

Feature Matrix: X = All medical attributes

Target Variable: y = target

```
model = LogisticRegression(max_iter = 1000)
model.fit(X_train,y_train)

y_pred = model.predict(X_test)
```

We are taking the Logistic Regression() for the model evalution from the mentioned Model buildings.

Sample:

```
print(y_test.values[:30])
print(y_pred[:30])
```

```
56  y_pred = model.predict(X_test)
57
58  print("Values of y_test values ",y_test.values[:30])
59  print("The predicted values:    ",y_pred[:30])
60
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS E:\Project_Code_Alpha> & "C:\Users\ADSS ABHISHEK\AppData\Local\Programs\Python\Python312\python.exe" e:/Project_Code_Alpha/Disease_prediction.py
Values of y_test values  [1 1 0 1 0 1 0 0 1 0 1 0 1 1 0 0 0 1 1 0 1 0 0 0 1 1 1 1 0 0]
The precited values:     [1 1 0 1 0 1 0 0 1 0 1 0 1 1 0 1 0 1 1 0 1 0 0 0 1 1 1 1 0 1]
```

from this output we can clearly see that there are some mismatch values from which the actual values to the predicted values: this shows we need to check the Model's Accuracy for the clear Picture.

```
61  print(accuracy_score(y_test, y_pred))
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS E:\Project_Code_Alpha> & "C:\Users\ADSS ABHISHEK\AppData\Local\Programs\Python\Python312\python.exe" e:/Project_Code_Alpha/Disease_prediction.py
0.7951219512195122
```

So our Model got the score of the 79.5% accuracy for the prediction of the Data.

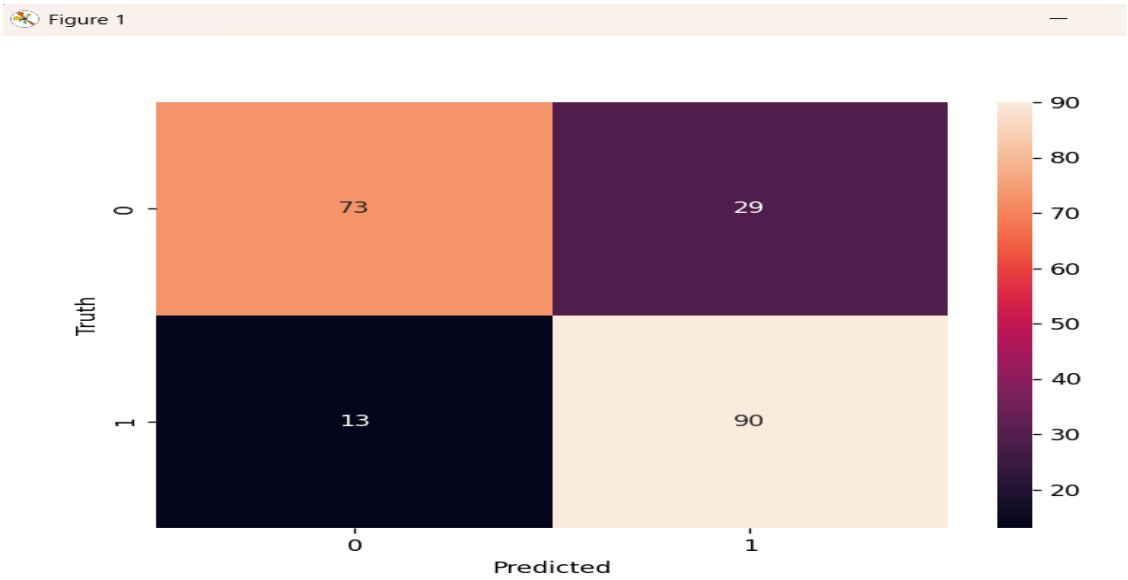
Confusion_matrix():

Disease_prediction.py

```
63  from sklearn.metrics import confusion_matrix
64  cm = confusion_matrix(y_test,y_pred)
65  print(cm)
66
67  plt.figure(figsize=(7,5))   # this part of the code is for the printing of the cm grpah clearly:
68  sn.heatmap(cm, annot=True)
69  plt.xlabel('Predicted')
70  plt.ylabel('Truth')
71  plt.show()
72
73  print(classification_report(y_test,y_pred))
74
75  sample = [[63,1,3,145,233,1,0,150,0,2.3,0,0,1]]
76  print(model.predict(sample))
77
78
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS E:\Project_Code_Alpha> & "C:\Users\ADSS ABHISHEK\AppData\Local\Programs\Python\Python312\python.exe" e:/Project_Code_Alpha/Disease_prediction.py
0.7951219512195122
[[73 29]
 [13 90]]
```



Classification Report(): we can clearly visualize the values of the Precision, accuracy, F1-score

```
print(classification_report(y_test,y_pred))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS E:\Project_Code_Alpha> & "C:\Users\ADSS ABHISHEK\AppData\Local\Programs\Python\Python312\python.exe" e:/Project_Code_Alpha/Disease_prediction.py
              precision    recall  f1-score   support

           0       0.85      0.72      0.78       102
           1       0.76      0.87      0.81       103

    accuracy                           0.80       205
   macro avg       0.80      0.79      0.79       205
weighted avg       0.80      0.80      0.79       205
```

Results and Evaluation

Metrics Used:

- Accuracy Score
- Confusion Matrix
- Precision
- Recall
- F1-Score

Model Accuracy:

79.5%

The Logistic Regression model achieved an accuracy of approximately 79.5% on the test dataset, indicating that the model can effectively classify patients with and without heart disease.

Prediction of Samples:

Sample1: Prediction: Heart Disease Present

```
sample = pd.DataFrame(
    [[63,1,3,145,233,1,0,150,0,2.3,0,0,1]],
    columns=X.columns
)
print("The predicted value of the model is: ", model.predict(sample))

prediction = model.predict(sample)
if prediction[0] == 1:
    print("Heart Disease Present")
else:
    print("No Heart Disease")
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS E:\Project_Code_Alpha> & "C:\Users\ADSS ABHISHEK\AppData\Local\Programs\Python\Python312\python.exe" e:/Project_Code_Alpha/Disease_prediction.py
[[73 29]
 [13 90]]
              precision    recall  f1-score   support

           0       0.85      0.72      0.78       102
           1       0.76      0.87      0.81       103

    accuracy                           0.80       205
   macro avg       0.80      0.79      0.79       205
weighted avg       0.80      0.80      0.79       205

The predicted value of the model is:  [1]
Heart Disease Present
PS E:\Project Code Alpha>
```

Sample2: Prediction: No Heart Disease

```
sample1 = pd.DataFrame(
    [[52,1,0,125,212,0,1,168,0,1.0,2,2,3]],
    columns=X.columns
)

print("The predicted value of the model is: ", model.predict(sample1))

prediction = model.predict(sample1)
if prediction[0] == 1:
    print("Heart Disease Present")
else:
    print("No Heart Disease")
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS E:\Project_Code_Alpha> & "C:\Users\ADSS ABHISHEK\AppData\Local\Programs\Python\Python312\python.exe" e:/Project_Code_Alpha/Disease_prediction.py
[[73 29]
 [13 90]]
              precision    recall  f1-score   support

           0       0.85      0.72      0.78       102
           1       0.76      0.87      0.81       103

    accuracy                           0.80       205
   macro avg       0.80      0.79      0.79       205
weighted avg       0.80      0.80      0.79       205

The predicted value of the model is:  [0]
No Heart Disease
```

Conclusion

This project successfully developed a Heart Disease Prediction System using Logistic Regression. The model was trained on a dataset containing 1025 patient records and achieved an accuracy of approximately 79.5%. The project demonstrates the practical application of machine learning in healthcare and can assist in early disease detection and support clinical decision-making.

References

1. UCI Machine Learning Repository – Heart Disease Dataset
2. Kaggle – Heart Disease Dataset
3. Scikit-Learn Documentation
4. Pandas Documentation
5. Matplotlib Documentation
6. Seaborn Documentation

Using the Random Forest Algorithm: 2nd method Implementation:

Heart Disease Prediction Using Random Forest Algorithm:

Abstract:

Random Forest, a supervised machine learning ensemble classification algorithm, was applied to predict the presence or absence of heart disease based on patient medical data.

Introduction:

Random Forest is an ensemble learning algorithm that combines multiple decision trees and uses majority voting to make predictions. It is widely used in healthcare applications because of its high accuracy and ability to handle complex datasets.

Methodology:

Heart Dataset

↓

Preprocessing

↓

Exploratory Data Analysis

↓

Feature Selection

↓

Train-Test Split

↓

Random Forest Classifier

↓

Prediction

↓

Model Evaluation

↓

Conclusion

Working of Random Forest:

Patient Medical Data

↓

Random Sampling of Data

↓

Build Multiple Decision Trees

↓

Each Tree Predicts:

Disease / No Disease

↓

Majority Voting

↓

Final Prediction

Model Building:

Algorithm Used:

Random Forest Classifier

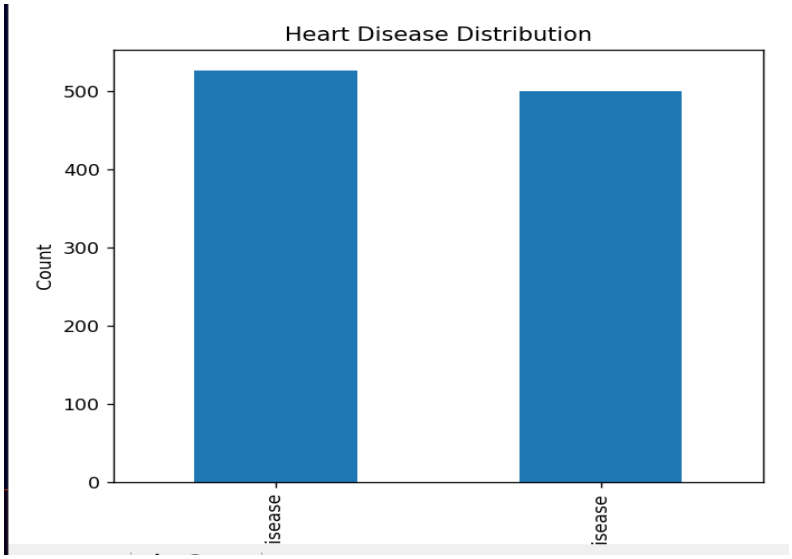
n_estimators = 100

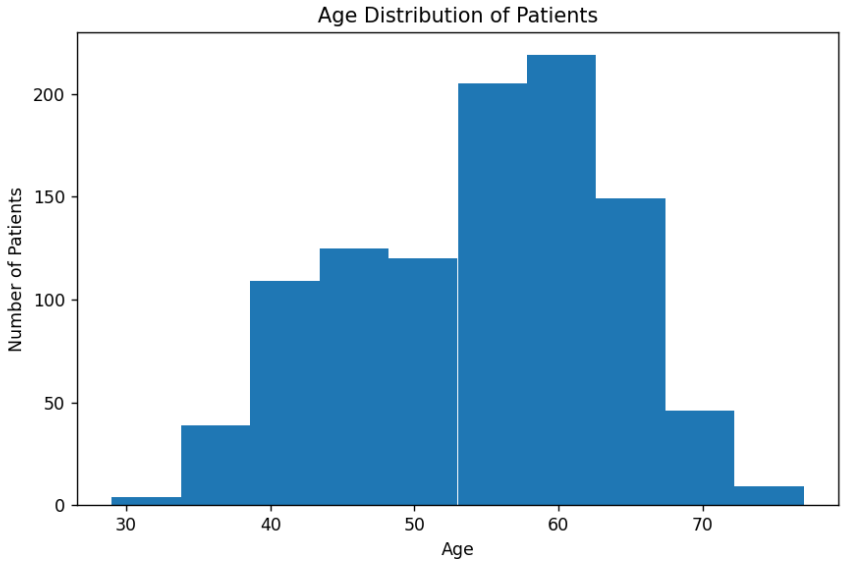
random_state = 42

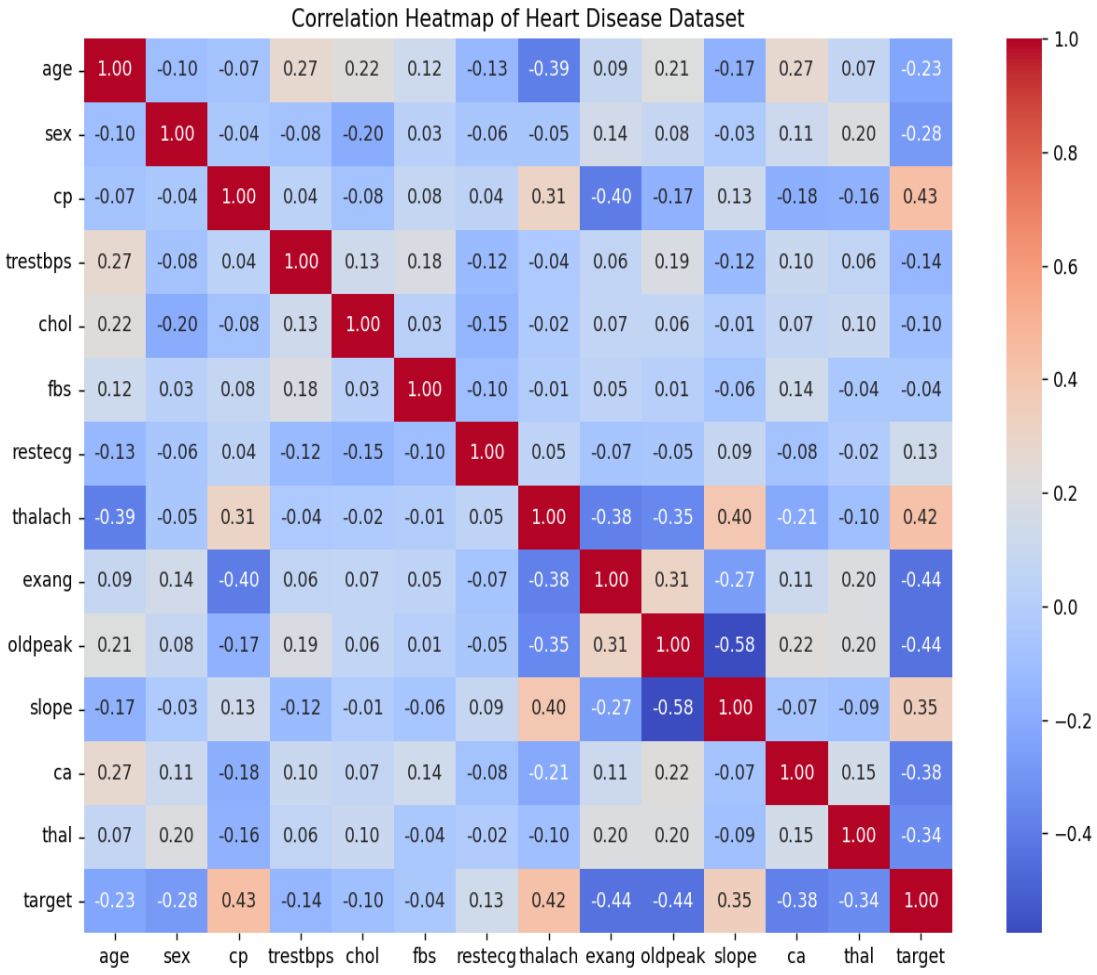
Result:

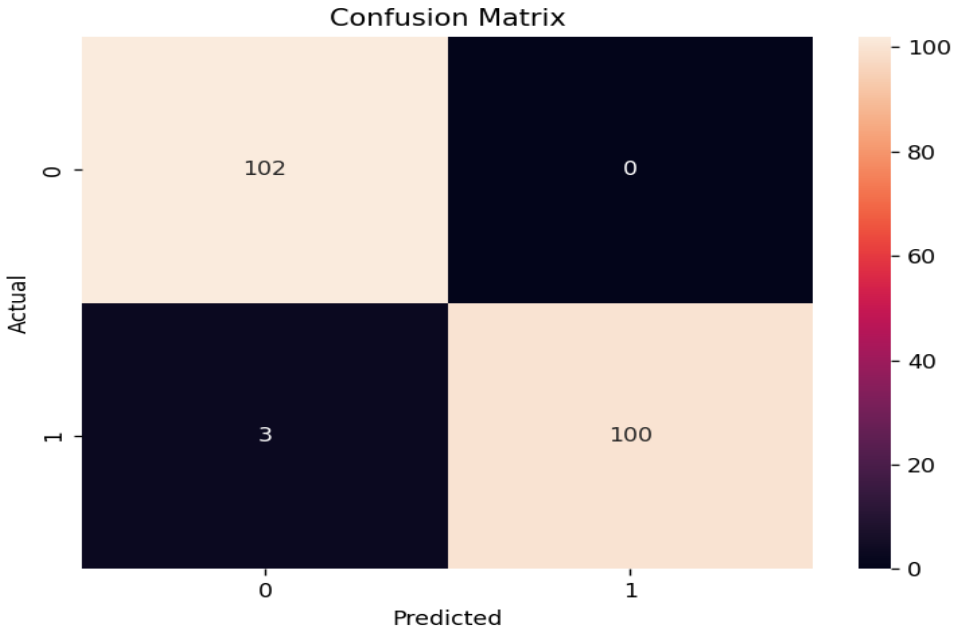
The Random Forest model achieved an accuracy of 98.5%.

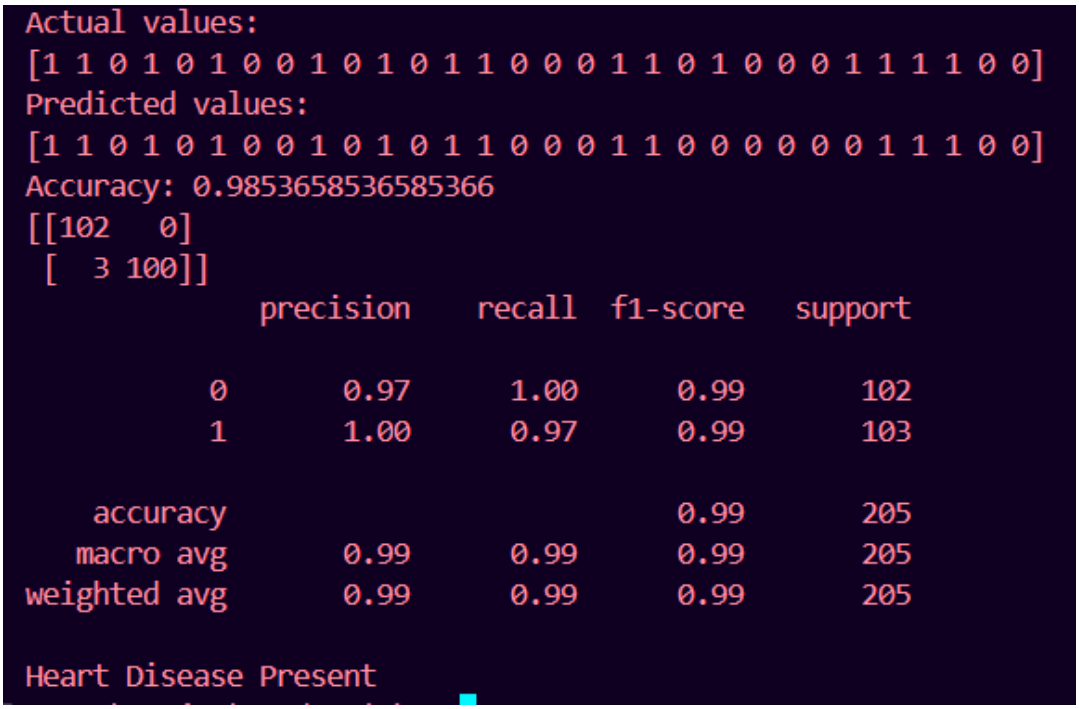
The model effectively classified patients with and without heart disease and demonstrated better predictive performance because it combines the decisions of multiple decision trees.






Correlation Heatmap of Heart Disease Dataset
age 1.00 -0.10 -0.07 0.27 0.22 0.12 -0.13 -0.39 0.09 0.21 -0.17 0.27 0.07 -0.23
sex -0.10 1.00 -0.04 -0.08 -0.20 0.03 -0.06 -0.05 0.14 0.08 -0.03 0.11 0.20 -0.28
cp -0.07 -0.04 1.00 0.04 -0.08 0.08 0.04 0.31 -0.40 -0.17 0.13 -0.18 -0.16 0.43
trestbps 0.27 -0.08 0.04 1.00 0.13 0.18 -0.12 -0.04 0.06 0.19 -0.12 0.10 0.06 -0.14
chol 0.22 -0.20 -0.08 0.13 1.00 0.03 -0.15 -0.02 0.07 0.06 -0.01 0.07 0.10 -0.10
fbs 0.12 0.03 0.08 0.18 0.03 1.00 -0.10 -0.01 0.05 0.01 -0.06 0.14 -0.04 -0.04
restecg -0.13 -0.06 0.04 -0.12 -0.15 -0.10 1.00 0.05 -0.07 -0.05 0.09 -0.08 -0.02 0.13
thalach -0.39 -0.05 0.31 -0.04 -0.02 -0.01 0.05 1.00 -0.38 -0.35 0.40 -0.21 -0.10 0.42
exang 0.09 0.14 -0.40 0.06 0.07 0.05 -0.07 -0.38 1.00 0.31 -0.27 0.11 0.20 -0.44
oldpeak 0.21 0.08 -0.17 0.19 0.06 0.01 -0.05 -0.35 0.31 1.00 -0.58 0.22 0.20 -0.44
slope -0.17 -0.03 0.13 -0.12 -0.01 -0.06 0.09 0.40 -0.27 -0.58 1.00 -0.07 -0.09 0.35
ca 0.27 0.11 -0.18 0.10 0.07 0.14 -0.08 -0.21 0.11 0.22 -0.07 1.00 0.15 -0.38
thal 0.07 0.20 -0.16 0.06 0.10 -0.04 -0.02 -0.10 0.20 0.20 -0.09 0.15 1.00 -0.34
target -0.23 -0.28 0.43 -0.14 -0.10 -0.04 0.13 0.42 -0.44 -0.44 0.35 -0.38 -0.34 1.00
age sex cp trestbps chol fbs restecg thalach exang oldpeak slope ca thal target
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4


Confusion Matrix
102
0
3
100
Actual
0
1
0
1
Predicted
100
80
60
40
20
0



Conclusion:

This project successfully developed a Heart Disease Prediction System using the Random Forest Classifier. The model was trained on a dataset containing 1025 patient records and achieved an accuracy of 98.5%. The results demonstrate that Random Forest can effectively predict heart disease and can assist healthcare professionals in early disease detection and clinical decision-making.

Algorithm	Accuracy
Logistic Regression	79.5%
Random Forest	98.5%

Random Forest outperformed Logistic Regression because it combines multiple decision trees and can capture complex relationships within medical data more effectively.